5. a) Bitmap checkpoint for object/window

Aim: To verify that a specific application window (or GUI object like a button or textbox) appears as expected by capturing and comparing it to a reference image.

Code:

```
import pyautogui
from PIL import ImageChops, Image
region = (100, 100, 200, 150)
screenshot = pyautogui.screenshot(region=region)
screenshot.save("current_object.png")
# Load reference image
reference = Image.open("reference_object.jpg")
diff=ImageChops.difference(screenshot, reference)
if diff.getbbox():
print("Bitmap checkpoint failed: Object/window has changed.")
else:
   print("Bitmapcheckpoint passed: Object/window is identical.")
```

output

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   SPELL CHECKER 5

Active code page: 65001

C:\Users\ashok\OneDrive\Desktop\stmlabprgs>python stmlab5a.py
Bitmap checkpoint failed: Object/window has changed.

C:\Users\ashok\OneDrive\Desktop\stmlabprgs>
```

5 b) Bitmap checkpoint for screen area

Aim: To validate that a certain area of the screen displays the expected content (e.g., dashboard, banner, etc.)

Code:

```
import pyautogui
from PIL import ImageChops, Image
region = (300, 200, 400, 300)
screenshot = pyautogui.screenshot(region=region)
screenshot.save("current_area.png")
reference = Image.open("reference_area.png")
diff=ImageChops.difference(screenshot, reference)
if diff.getbbox():
    print("Bitmap checkpoint failed: Screen area has
changed.")
else:
    print("Bitmap checkpoint passed: Screen area is
identical.")
```

output:

```
PROBLEMS 2   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   SPELL CHECKER 9

Active code page: 65001

C:\Users\ashok\OneDrive\Desktop\stmlabprgs>python stmlab5b.py
Bitmap checkpoint failed: Screen area has changed.

C:\Users\ashok\OneDrive\Desktop\stmlabprgs>
```

6)Database checkpoint for Default check

Aim: To validate that a specific column in a table has a default value, and when a new row is inserted without specifying that column, the default is applied.

Code:

```
import sqlite3
conn = sqlite3.connect(":memory:")
cursor = conn.cursor()
# Step 2: Create a table with a column that has a
DEFAULT value
cursor.execute('''
CREATE TABLE users (id INTEGER PRIMARY KEY
AUTOINCREMENT,name TEXT,
    status TEXT DEFAULT 'active')''')
cursor.execute("INSERT INTO users (name) VALUES
('Ashok')")
cursor.execute("SELECT name, status FROM users
WHERE name = 'Ashok'")
row = cursor.fetchone()
if row[1] == 'active':
    print("Database checkpoint passed: Default
value applied correctly.")
else:
    print("Database checkpoint failed: Default value
not applied.")
conn.close()
```

output:

```
PROBLEMS 2   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   SPELL CHECKER 11

Active code page: 65001

C:\Users\ashok\OneDrive\Desktop\stmlabprgs>python stmlab6.py
Database checkpoint passed: Default value applied correctly.

C:\Users\ashok\OneDrive\Desktop\stmlabprgs>
```

7) Database checkpoint for custom check

Aim: Verify a custom condition in the database and raise a checkpoint pass/fail based on the logic.

Code:

```
import sqlite3
conn = sqlite3.connect(":memory:")
cursor = conn.cursor()
cursor.execute('''CREATE TABLE users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    email TEXT,
    status TEXT)''')
cursor.executemany("INSERT INTO users (name,
email, status) VALUES (?, ?, ?)", [
    ("Ashok", "ashok@example.com", "active"),
    ("Bob", "bob@example.com", "inactive"),
    ("Chandhu", "chandhu@example.com",
"active")])
cursor.execute('''
SELECT id, name, email, status
FROM users
WHERE email IS NULL OR status NOT IN ('active',
'inactive')''')
violations = cursor.fetchall()
if violations:
    print("Database checkpoint failed: Found invalid
user records.")
```

```
  for row in violations:
print(f" - ID {row[0]}: {row[1]} | Email: {row[2]} |
Status: {row[3]}")
else:
print("Database checkpoint passed: All user
records are valid.")
conn.close()
```

output:

```
PROBLEMS 2   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   SPELL CHECKER 15

Active code page: 65001

C:\Users\ashok\OneDrive\Desktop\stmlabprgs>python stmlab7.py
Database checkpoint passed: All user records are valid.

C:\Users\ashok\OneDrive\Desktop\stmlabprgs>
```

8) Database checkpoint for runtime record check

Aim: To validate database records at runtime by checking if they meet predefined business rules or constraints**.**

Code:

```
import sqlite3
# Step 1: Connect to database
conn = sqlite3.connect(":memory:")
cursor = conn.cursor()
# Step 2: Create table
cursor.execute('''
CREATE TABLE users (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   name TEXT,
```

```
    email TEXT,
    status TEXT )''')
new_user =("chandhu","chandhu@example.com",
"active")
cursor.execute("INSERT INTO users (name, email,
status) VALUES (?, ?, ?)", new_user)
cursor.execute('''
SELECT id, name, email, status
FROM users
WHERE name = ? AND email = ? AND status IN
('active', 'inactive')
''', (new_user[0], new_user[1]))
record = cursor.fetchone()
if record:
    print("Database checkpoint passed: Runtime
record check successful.")
 print(f" - Found user: ID {record[0]}, Name:
{record[1]}, Email: {record[2]}, Status: {record[3]}")
else:
    print("Database checkpoint failed: Expected
runtime record not found or invalid.")
conn.close()
```

Output:

```
PROBLEMS 2   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   SPELL CHECKER 15

Active code page: 65001

C:\Users\ashok\OneDrive\Desktop\stmlabprgs>python stmlab8.py
Database checkpoint passed: Runtime record check successful.
 - Found user: ID 1, Name: chandhu, Email: chandhu@example.com, Status: active

C:\Users\ashok\OneDrive\Desktop\stmlabprgs>
```

9 a) Data driven test for dynamic test data submission

Aim : To automate test execution by dynamically submitting multiple sets of test data to verify application behavior under varying input conditions.

Code

```
import sqlite3
test_users = [  {"name": "Ashok", "email":
"ashok@example.com", "status": "active"},
   {"name": "Bob", "email": "bob@example.com",
"status": "inactive"},
   {"name": "Chandhu", "email":
"charlie@example.com", "status": "active"},
   {"name": "uma", "email": None, "status":
"active"},
conn = sqlite3.connect(":memory:")
cursor = conn.cursor()
cursor.execute('''CREATE TABLE users (id INTEGER
PRIMARY KEY AUTOINCREMENT,
   name TEXT,
   email TEXT NOT NULL,
   status TEXT CHECK(status IN ('active',
'inactive')))''')
for user in test_users:
   try:
      cursor.execute(
         "INSERT INTO users (name, email, status)
VALUES (?, ?, ?)",
         (user["name"], user["email"],
user["status"])
      )
      print(f" Test passed: Inserted user
{user['name']}")
   except Exception as e:
```

```
  print(f" Test failed: Could not insert user
{user['name']} — {e}")
cursor.execute("SELECT COUNT(*) FROM users")
count = cursor.fetchone()[0]
print(f"\n Total successful inserts: {count} out of
{len(test_users)}")
conn.close()
```

output:

```
PROBLEMS 2   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   SPELL CHECKER 17

Active code page: 65001

C:\Users\ashok\OneDrive\Desktop\stmlabprgs>python stmlab9a.py
 Test passed: Inserted user Ashok
 Test passed: Inserted user Bob
 Test passed: Inserted user Chandhu
 Test failed: Could not insert user uma — NOT NULL constraint failed: users.email

 Total successful inserts: 3 out of 4

C:\Users\ashok\OneDrive\Desktop\stmlabprgs>
```

9 b) Data driven test through flat files

Aim: To automate testing of an application by using flat files (such as CSV or TXT) as input data sources, enabling dynamic and data-driven validation of various functionalities based on different sets of test data.

Code:

```
import sqlite3
import csv
conn = sqlite3.connect(":memory:")
cursor = conn.cursor()
cursor.execute('''
CREATE TABLE users (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
    name TEXT NOT NULL,
    email TEXT NOT NULL,
    status TEXT CHECK(status IN ('active',
'inactive')))''')
with open('test_data.csv', newline='') as csvfile:
    reader = csv.DictReader(csvfile)
    success_count = 0
    total = 0

    for row in reader:
        total += 1
        try:
            cursor.execute(
                "INSERT INTO users (name, email, status)
VALUES (?, ?, ?)",
                (row["name"], row["email"],
row["status"])
            )
            print(f" Test passed: Inserted user
{row['name']}")
            success_count += 1
        except Exception as e:
            print(f" Test failed: Could not insert user
{row['name']} — {e}")
            print("CSV Headers:", reader.fieldnames)

  print(f"\nSummary: {success_count} out of {total}
records inserted successfully.")
conn.close()
```

output:

```
PROBLEMS 2   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   SPELL CHECKER 5

Active code page: 65001

C:\Users\ashok\OneDrive\Desktop\stmlabprgs>python stmlab9b.py
 Test passed: Inserted user Ashok
 Test passed: Inserted user Bob
 Test passed: Inserted user Chandhu

Summary: 3 out of 3 records inserted successfully.

C:\Users\ashok\OneDrive\Desktop\stmlabprgs>
```

9 c) Data driven test through front grids

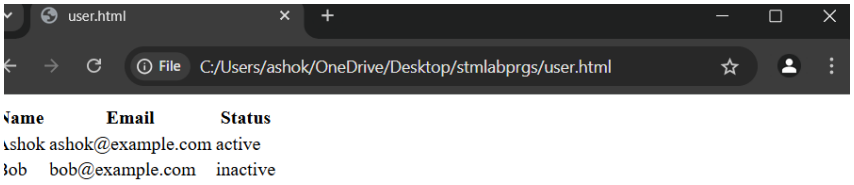
Aim: To validate application functionality by driving test inputs from front-end data grids and verifying the results dynamically.

Code

```
import csv
from selenium import webdriver
from selenium.webdriver.common.by import By
import time
expected_data = []
with open("test_data.csv", newline='') as f:
    reader = csv.DictReader(f)
    expected_data = list(reader)
driver = webdriver.Chrome()  # make sure
chromedriver is installed
driver.get("file:///C:/Users/ashok/OneDrive/Deskt
op/stmlabprgs/user.html")  # Or use a live URL
time.sleep(2)
actual_data = []
rows = driver.find_elements(By.CSS_SELECTOR,
"#userTable tbody tr")
for row in rows:
    cols = row.find_elements(By.TAG_NAME, "td")
```

```
    actual_data.append({
        "name": cols[0].text.strip(),
        "email": cols[1].text.strip(),
        "status": cols[2].text.strip() })
# Step 4: Compare actual vs expected
for expected in expected_data:
    match = False
    for actual in actual_data:
        if all(expected[k].strip().lower() ==
actual[k].strip().lower() for k in expected):
            match = True
            break
    if match:
        print(f"PASS: {expected}")
    else:
        print(f" FAIL: {expected}")
driver.quit()
```

Output:



9 d) Data driven test through excel test

Aim: To automate testing by reading multiple sets of input data from an Excel file and validating application behavior based on the provided test scenarios.

Code

```
import sqlite3
import openpyxl
wb = openpyxl.load_workbook('test_users.xlsx')
sheet = wb.active
conn = sqlite3.connect(":memory:")
cursor = conn.cursor()
cursor.execute('''
CREATE TABLE users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT NOT NULL,
    status TEXT CHECK(status IN ('active',
'inactive')))''')
success_count = 0
total = 0
# Skip header row
for row in sheet.iter_rows(min_row=2,
values_only=True):
    name, email, status = row
    total += 1
    try:
        cursor.execute( "INSERT INTO users (name,
email, status) VALUES (?, ?, ?)", (name, email,
status) )
        print(f" Test passed: Inserted user {name}")
        success_count += 1
    except Exception as e:
```

```
    print(f" Test failed: Could not insert user {name}
— {e}")
print(f"\nSummary: {success_count} out of {total}
records inserted successfully.")
conn.close()
```

output:

```
PROBLEMS 2   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   SPELL CHECKER 9

Active code page: 65001

C:\Users\ashok\OneDrive\Desktop\stmlabprgs>python stmlab9d.py
 Test passed: Inserted user Ashok
 Test passed: Inserted user Bob
 Test failed: Could not insert user Chandhu — NOT NULL constraint failed: users.email
 Test failed: Could not insert user uma — CHECK constraint failed: status IN ('active', 'inactive')

Summary: 2 out of 4 records inserted successfully.
```

10 a) Batch testing without parameter passing

Aim: To execute a group of predefined test cases sequentially without passing external parameters

Code

```
def validate_user(name, email, status):
    if not email:
        return False
    if status not in ['active', 'inactive']:
        return False
    return True

test_cases = [ ("Ashok", "Ashok@example.com",
"active"), ("Bob", None, "inactive"),
    ("Chandhu", "chandhu@example.com",
"banned"),  ("Dhana", "dhana@example.com",
"active")]
for i, (name, email, status) in
enumerate(test_cases, 1):
    result = validate_user(name, email, status)
    if result:
        print(f"Test {i} Passed: {name}")
```

```
    else:
        print(f" Test {i} Failed: {name}")
```

Output:

```
PROBLEMS 2   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   SPELL CHECKER 12

Active code page: 65001

C:\Users\ashok\OneDrive\Desktop\stmlabprgs>python stmlab10a.py
Test 1 Passed: Ashok
 Test 2 Failed: Bob
 Test 3 Failed: Chandhu
Test 4 Passed: Dhana
```

10 b) Batch testing with parameter passing

Aim: To execute a group of predefined test cases sequentially with passing external parameters

Code

```
import csv
def validate_user(name, email, status):
    if not email:
        return False
    if status not in ['active', 'inactive']:
        return False
    return True

# Read test cases from CSV
with open('test_cases.csv', newline='') as file:
    reader = csv.DictReader(file)
    for i, row in enumerate(reader, 1):
        name = row["name"]
        email = row["email"]
        status = row["status"]
        result = validate_user(name, email, status)
        if result:
```

```
        print(f"Test {i} Passed: {name}")
    else:
        print(f" Test {i} Failed: {name}")
```

Output:

```
PROBLEMS 2   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   SPELL CHECKER 12

Active code page: 65001

C:\Users\ashok\OneDrive\Desktop\stmlabprgs>python stmlab10b.py
Test 1 Passed: Ashok
 Test 2 Failed: Bob
 Test 3 Failed: Chandhu
Test 4 Passed: Dhana
```

11) Data driven batch

Aim: To perform batch execution of test cases using external test data sources, enabling efficient validation of application functionality across multiple data sets.

Code

```
import csv
# Define validation function
def validate_user(name, email, status):
    if not email:
        return False
    if status not in ['active', 'inactive']:
        return False
    return True
# Step 1: Load data from CSV
with open('test_cases.csv', newline='') as file:
    reader = csv.DictReader(file)
    total = 0
    passed = 0
```

```
for i, row in enumerate(reader, 1):
        name = row['name']
        email = row['email']
        status = row['status']
        total += 1
  # Step 2: Execute validation
        if validate_user(name, email, status):
            print(f" Test {i} Passed: {name}")
            passed += 1
        else:
            print(f" Test {i} Failed: {name}")

print(f"\nSummary: {passed} out of {total} tests
passed.")
```

output:

```
PROBLEMS 2   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   SPELL CHECKER 12

Active code page: 65001

C:\Users\ashok\OneDrive\Desktop\stmlabprgs>python stmlab11.py
 Test 1 Passed: Ashok
 Test 2 Failed: Bob
 Test 3 Failed: Chandhu
 Test 4 Passed: Dhana

Summary: 2 out of 4 tests passed.
```

12) Silent mode test execution without any interruption

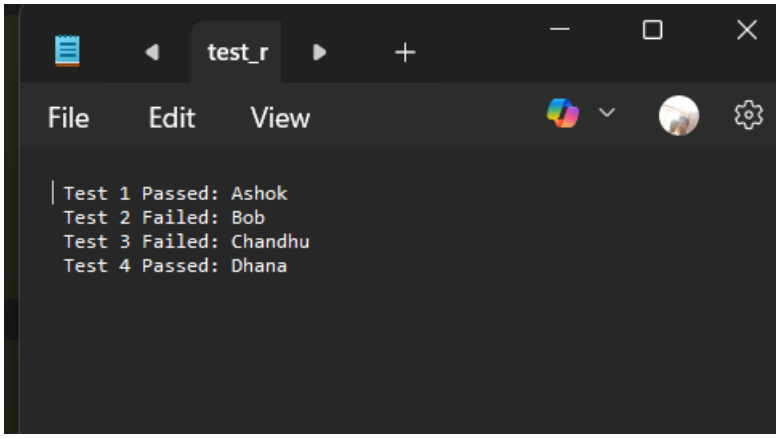
Aim: To execute test cases in silent mode without any user interaction or interruptions, ensuring smooth and automated test runs.

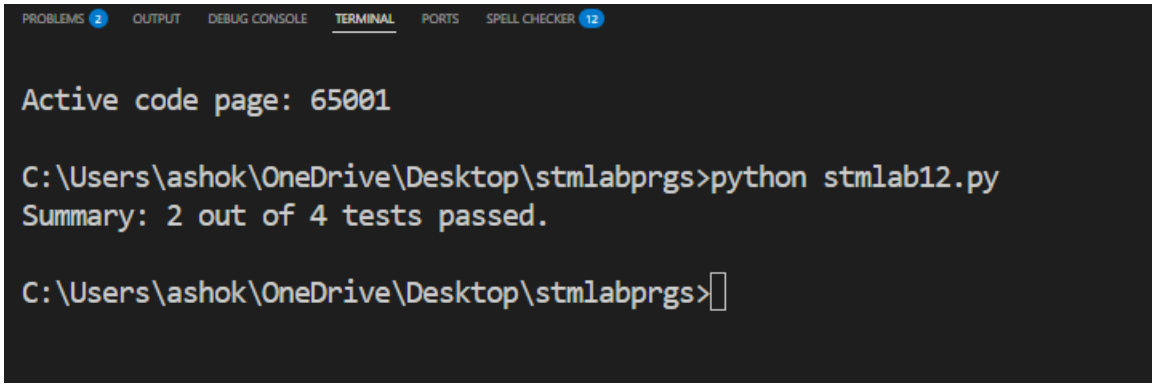
Code

```
import csv
import os
import sys
def validate_user(name, email, status):
    if not email:
        return False
    if status not in ['active', 'inactive']:
        return False
    return True
sys.stdout = open(os.devnull, 'w')
with open('test_cases.csv', newline='') as file:
    reader = csv.DictReader(file)
    total = 0
    passed = 0
    with open("test_results.log", "w") as log_file:
        for i, row in enumerate(reader, 1):
            name = row['name']
            email = row['email']
            status = row['status']
            total += 1
            if validate_user(name, email, status):
                log_file.write(f" Test {i} Passed:
{name}\n")
                passed += 1
            else:
                log_file.write(f" Test {i} Failed: {name}\n")
sys.stdout = sys.__stdout__  # Enable console
output again
print(f"Summary: {passed} out of {total} tests
passed.")
```

output:

automatically the log file as been generated as shown as :





13) Test case for calculator in windows application

Aim: To validate the functional correctness of the Windows Calculator application by automating arithmetic operations

Code

```
from pywinauto import Application
from time import sleep
def test_calculator_addition():
    # Step 1: Launch Calculator
  app =Application(backend="uia").start("calc.exe")
    # Step 2: Connect to the Calculator window
 calc_window = app.window(title_re="Calculator")
    calc_window.wait("visible", timeout=10)
calc_window.child_window(title="Seven",
control_type="Button").click()
```

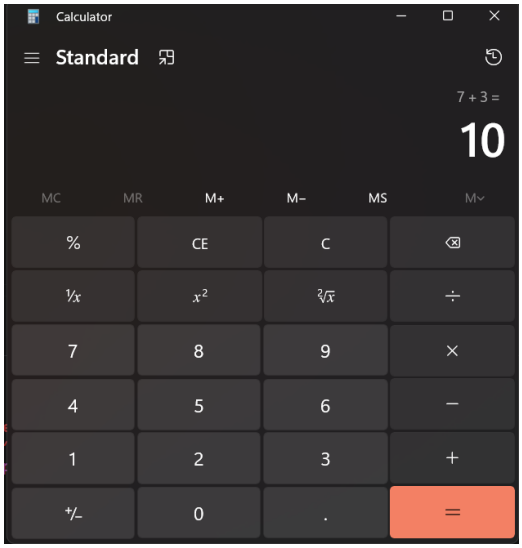
```
    calc_window.child_window(title="Plus",
control_type="Button").click()
    calc_window.child_window(title="Three",
control_type="Button").click()
    calc_window.child_window(title="Equals",
control_type="Button").click()
 sleep(1)
    result_element =
calc_window.child_window(auto_id="CalculatorRe
sults", control_type="Text")
    result_text = result_element.window_text()



    print(f" Raw result text: '{result_text}'")

    result = result_text.replace("Display is",
"").strip()
if result == "10":
        print("Test Passed: 7 + 3 = 10")
    else:
        print(f" Test Failed: Expected 10 but got
'{result}'")
    calc_window.close()
if __name__ == "__main__":
    test_calculator_addition()
```

output:


Calculator
Standard
7 + 3 =
10
MC
MR
M+
M−
MS
M∨
%
CE
C
1/x
x²
∛x
÷
7
8
9
×
4
5
6
−
1
2
3
+
+/−
0
.
=